

도서 대여·반납 시스템



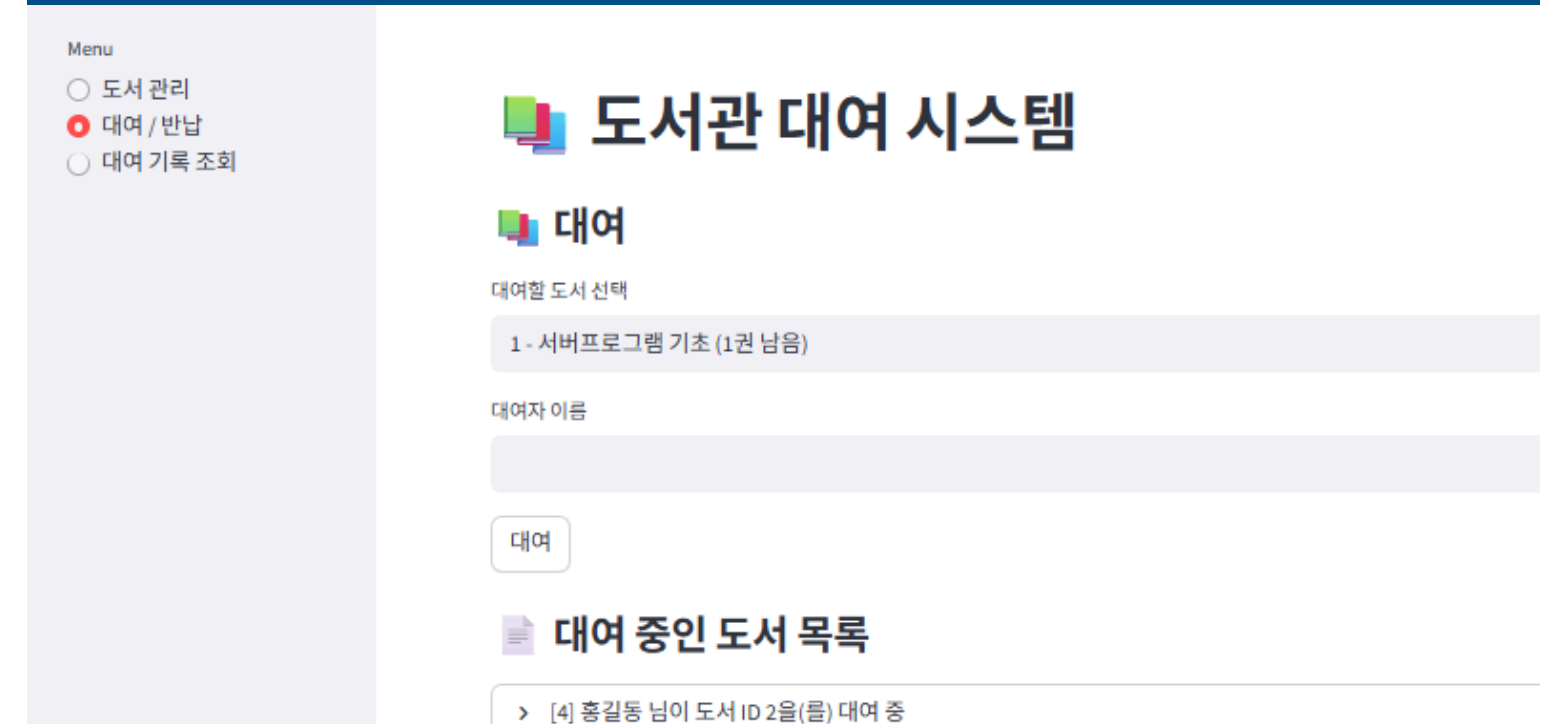
default

GET	/books	Read Books
POST	/books	Create Book
GET	/books/{book_id}	Read Book
PUT	/books/{book_id}	Update Book
DELETE	/books/{book_id}	Delete Book
POST	/loans/borrow	Borrow Book
POST	/loans/{loan_id}/return	Return Book
GET	/loans	Read Loans

본 시스템은 다음 기능을 포함

- 도서 CRUD (등록 · 조회 · 수정 · 삭제)
- 도서 대여
- 도서 반납
- 남은 재고 관리
- 대여/반납 이력 조회
- Streamlit 기반의 간단한 웹 UI 제공

프로젝트 목표



프로젝트 구조

backend/

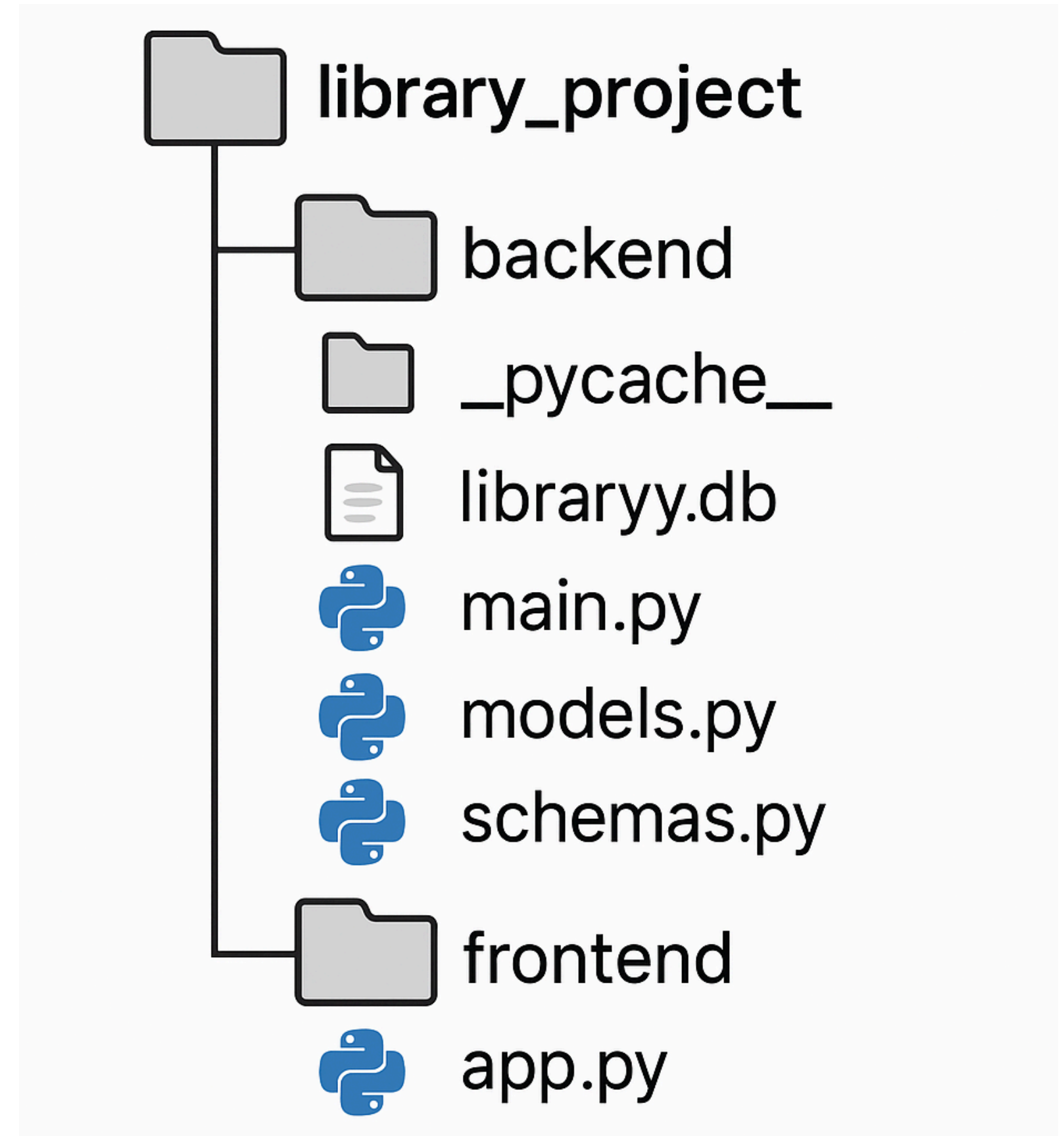
FastAPI 기반 서버(Backend) 코드가 있는 폴더와 데이터 처리 · API 제공 · DB 작업 담당

- **database.py**
- **models.py**
- **schemas.py**
- **main.py**

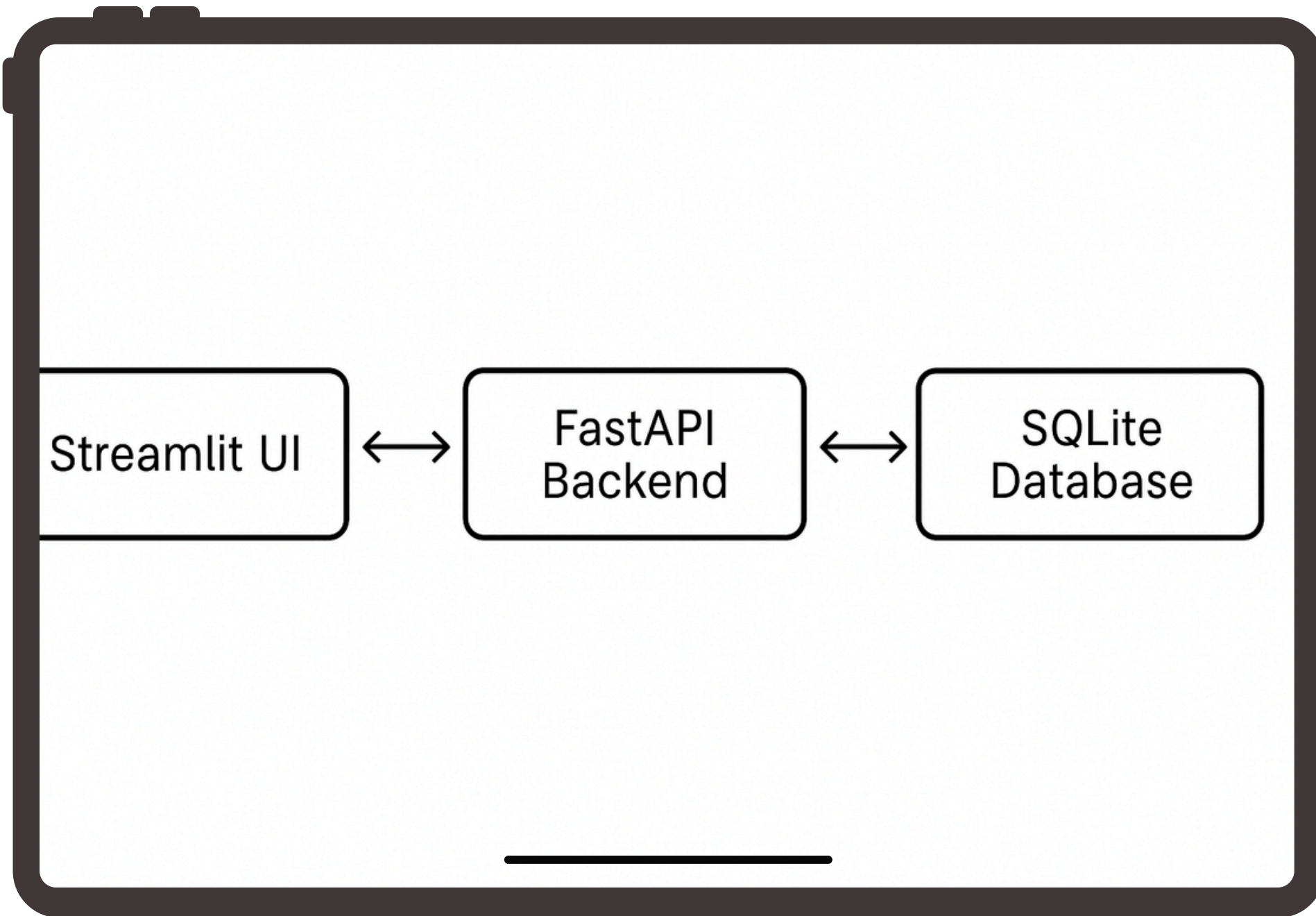
frontend/

Streamlit UI 코드가 있는 폴더와 사용자가 책 등록·조회·대여·반납을 수행하는 화면 구현

- **app.py**: Streamlit 화면 구성하고 Backend API 호출하여 기능 동작



전체 아키텍처



본 시스템은 *Streamlit*, *FastAPI*, *SQLite*로 구성되어 있다

- **Streamlit**은 사용자 인터페이스를 담당
- **FastAPI**는 비즈니스 로직 처리와 API 제공을 맡는다.
- **SQLite**는 모든 도서 및 대여 정보를 저장하는 데이터베이스 역할



DATABASE.PY 구성

```
# Cấu hình Database
SQLALCHEMY_DATABASE_URL = "sqlite:///./backend/library.db"

# Khởi tạo Engine tạo kết nối đến database
engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)

# Khởi tạo Session Local
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Khởi tạo Base cho các mô hình SQLAlchemy
Base = declarative_base()

# Dependency: Hàm để lấy và đóng session DB
def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

database.py는 DB 연결을 위한 핵심 파일입니다.

1. Engine 생성

- create_engine을 이용해 SQLite와 연결하는 엔진을 생성한다.

2. Base 선언

- declarative_base를 통해 모델 (Book, Loan)을 정의할 때 사용할 Base 클래스를 만든다

3. SessionLocal 생성

- SessionLocal은 요청마다 독립적인 DB 세션을 생성하도록 해 주는 객체

4. get_db() 의존성

- get_db() 의존성을 통해 FastAPI가 각 요청에서 세션을 열고, 요청이 끝나면 자동으로 닫도록 관리해 준다.

데이터베이스 설계 (MODELS.PY)

데이터베이스는 books 테이블과
loans 테이블 두 가지로 구성된다

```
class Book(Base):
    __tablename__ = "books"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True, nullable=False)
    author = Column(String, nullable=False)
    total_copies = Column(Integer, nullable=False, default=1)
    available_copies = Column(Integer, nullable=False, default=1)

    loans = relationship("Loan", back_populates="book")
```

books 테이블

- id
- title
- author
- total_copies
- available_copies

loans 테이블

- id
- book_id (Book.id 참조)
- borrower_name
- borrow_date
- return_date
- is_returned

```
class Loan(Base):
    __tablename__ = "loans"

    id = Column(Integer, primary_key=True, index=True)
    book_id = Column(Integer, ForeignKey("books.id"), nullable=False)
    borrower_name = Column(String, nullable=False)
    borrow_date = Column(DateTime, default=datetime.utcnow)
    return_date = Column(DateTime, nullable=True)
    is_returned = Column(Boolean, default=False)

    book = relationship("Book", back_populates="loans")
```

두 테이블은 book_id를 통해 연결되며,
하나의 Book은 여러 Loan을 가질 수 있지만
하나의 Loan은 한 Book에만 속한다.

스키마 (SCHEMAS.PY)

스키마는 *Pydantic*으로 작성되며 *API* 입·출력 데이터 구조를 정의한다.

- 스키마를 사용하면 입력 데이터 검증이 자동으로 이루어지고,
- *orm_mode*와 *from_attributes* 설정을 통해 *SQLAlchemy* 모델을 그대로 *API* 응답으로 변환할 수 있다

Book 관련 스키마

```
# ----- Book -----
class BookBase(BaseModel):
    title: str
    author: str
    total_copies: int = 1
class BookCreate(BookBase):
    pass
class BookUpdate(BaseModel):
    title: str | None = None
    author: str | None = None
    total_copies: int | None = None
class BookOut(BookBase):
    id: int
    available_copies: int

class Config:
    orm_mode = True
```

- BookBase: 기본 필드(title, author, total_copies)
- BookCreate: 도서 등록용
- BookUpdate: 도서 수정용 (부분 수정 가능)
- BookOut: 응답(Response)용 (id, available_copies 포함)

Loan 관련 스키마

- LoanCreate: 대여 요청
- LoanOut: 대여/반납 이력 응답

스키마를 사용하면 입력 데이터 검증 및 깔끔한 API 응답이 가능합니다.

```
# ----- Loan -----
class LoanCreate(BaseModel):
    book_id: int
    borrower_name: str

class LoanOut(BaseModel):
    id: int
    book_id: int
    borrower_name: str
    borrow_date: datetime
    return_date: datetime | None
    is_returned: bool

class Config:
    from_attributes = True
```

주요 API

이 표는 FastAPI에서 구현된 Book API와 Loan API의 메서드를 정리한 것입니다

 Book API		
Method	Endpoint	기능
GET	/books	도서 조회
POST	/books	도서 등록
PUT	/books/{id}	도서 수정
DELETE	/books/{id}	도서 삭제

왼쪽 Book API는 GET, POST, PUT, DELETE 네 가지 메서드로 도서 조회, 등록, 수정, 삭제 기능을 제공한다

 Loan API		
Method	Endpoint	기능
POST	/loans/borrow	도서 대여
POST	/loans/{id}/return	도서 반납
GET	/loans/history	대여·반납 이력 조회

오른쪽 Loan API는 POST와 GET 메서드를 사용하여 도서 대여, 반납, 그리고 대여·반납 이력 조회 기능을 제공합니다.

시스템 운영 로직

API 실행

1. FastAPI 앱 생성: `app = FastAPI(title="Library Rental System")`
2. `Base.metadata.create_all(bind=engine)`로 Book/Loan 테이블 자동 생성
3. `Depends(get_db)`를 통해 모든 엔드포인트에서 DB 세션 의존성 주입한다

```
from . import models, schemas
from .database import engine, Base, get_db
```

```
# Tạo bảng
```

```
Base.metadata.create_all(bind=engine)
```

```
app = FastAPI(title="Library Rental System")
```

```
def create_book(book: schemas.BookCreate, db: Session = Depends(get_db)):
```

시스템 운영 로직

📖 도서 등록 로직

1. 사용자가 입력한 도서 정보(title, author, total_copies)를 전달받는다
2. 전달받은 값으로 새로운 Book 객체를 생성한다
3. DB에 Book 객체를 추가하고 commit으로 저장한다
4. refresh로 최신 상태를 불러온 뒤 생성된 도서 정보를 반환한다

```
# ----- BOOK CRUD -----  
@app.post("/books", response_model=schemas.BookOut)  
def create_book(book: schemas.BookCreate, db: Session = Depends(get_db)):  
    db_book = models.Book(  
        title=book.title,  
        author=book.author,  
        total_copies=book.total_copies,  
        available_copies=book.total_copies,  
    )  
    db.add(db_book)  
    db.commit()  
    db.refresh(db_book)  
    return db_book
```


시스템 운영 로직

```
@app.put("/books/{book_id}", response_model=schemas.BookOut)
def update_book(
    book_id: int,
    book_update: schemas.BookUpdate,
    db: Session = Depends(get_db),
):
    book = db.query(models.Book).get(book_id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")

    if book_update.title is not None:
        book.title = book_update.title
    if book_update.author is not None:
        book.author = book_update.author
    if book_update.total_copies is not None:
        # điều chỉnh available khi total thay đổi
        diff = book_update.total_copies - book.total_copies
        book.total_copies = book_update.total_copies
        book.available_copies += diff
        if book.available_copies < 0:
            book.available_copies = 0
```

도서 수정 로직

1. book_id로 수정할 도서가 존재하는지 조회한다
2. 전달받은 값(title, author, total_copies) 중 None이 아닌 항목만 업데이트한다
3. total_copies가 변경된 경우, 차이(diff)에 따라 available_copies를 조정한다
4. 업데이트를 commit하고 최신 상태를 반환한다

시스템 운영 로직

도서 대여 로직

1. 사용자가 요청한 책이 실제로 존재하는지 확인
2. 해당 책의 대여 가능 권수가 남아 있는지 검사
3. 조건이 모두 충족되면 새로운 Loan 기록을 생성
4. 대여가 완료되면 해당 도서의 available_copies 값을 1 감소시킨다

```
def borrow_book(loan: schemas.LoanCreate, db: Session = Depends(get_db)):
    book = db.query(models.Book).get(loan.book_id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")

    if book.available_copies <= 0:
        raise HTTPException(status_code=400, detail="No available copies")

    db_loan = models.Loan(
        book_id=loan.book_id,
        borrower_name=loan.borrower_name,
    )
    book.available_copies -= 1
```

시스템 운영 로직

📖 도서 반납 로직

1. 반납하려는 Loan 기록이 실제로 존재하는지 조회한다
2. return_date를 현재 시간으로 저장한다
3. is_returned 값을 True로 변경해 반납 상태로 표시한다
4. 도서의 available_copies를 1 증가시켜 재고를 복구한다

```
@app.post("/loans/{loan_id}/return", response_model=schemas.LoanOut)
def return_book(loan_id: int, db: Session = Depends(get_db)):
    loan = db.query(models.Loan).get(loan_id)
    if not loan:
        raise HTTPException(status_code=404, detail="Loan not found")
    if loan.is_returned:
        raise HTTPException(status_code=400, detail="Already returned")

    loan.is_returned = True
    from datetime import datetime

    loan.return_date = datetime.utcnow()

    # tăng lại số lượng sách
    book = db.query(models.Book).get(loan.book_id)
    if book:
        book.available_copies += 1
```


시스템 운영 로직

도서 삭제

도서 삭제는 Loan 기록이 없는 경우에만 가능

- 먼저 요청한 도서가 실제로 존재하는지 확인하고, 그 다음 해당 도서에 대여 기록이 있는지 검사한다.
- 대여 기록이 하나라도 있으면 삭제할 수 없고 오류 메시지를 반환한다.
- 기록이 없다면 도서를 안전하게 삭제하고 성공 메시지를 돌려준다.

```
@app.delete("/books/{book_id}")
def delete_book(book_id: int, db: Session = Depends(get_db)):
    # 1. Tìm sách
    book = db.query(models.Book).filter(models.Book.id == book_id).first()
    if not book:
        raise HTTPException(status_code=404, detail="책을 찾을 수 없습니다.") # Không tìm thấy sách

    # 2. Kiểm tra xem sách có từng được mượn chưa
    has_loan = db.query(models.Loan).filter(models.Loan.book_id == book_id).first()
    if has_loan:
        raise HTTPException(
            status_code=400,
            detail="이미 대여 기록이 있어서 삭제할 수 없습니다." # Đã có lịch sử mượn nên không xóa
        )

    # 3. Xóa sách
    db.delete(book)
    db.commit()
    return {"message": "도서를 삭제했습니다."}
```

STREAMLIT UI 구성

- Streamlit에서 도서명·저자·총 권수를 입력받고 ‘추가’ 버튼을 누르면, 입력값을 JSON으로 FastAPI의 POST /books 엔드포인트에 전송한다.
- FastAPI는 받은 데이터를 기반으로 새로운 Book 객체를 생성하여 SQLite DB에 저장하고, 성공 시 Streamlit에 성공 메시지를 반환한다.

도서 추가

도서명

저자

총 권수

추가

```
# ----- Quản lý sách -----
```

```
if menu == "도서 관리":
```

```
    st.subheader("📖 도서 추가")
```

```
    with st.form("add_book_form"):
```

```
        title = st.text_input("도서명")
```

```
        author = st.text_input("저자")
```

```
        total = st.number_input("총 권수", min_value=1, value=1, step=1)
```

```
        submitted = st.form_submit_button("추가")
```

```
if submitted:
```

```
    payload = {"title": title, "author": author, "total_copies": total}
```

```
    r = requests.post(f"{BACKEND_URL}/books", json=payload)
```

```
    if r.status_code == 200:
```

```
        st.success("추가됨")
```

```
    else:
```

```
        st.error(f"오류: {r.text}")
```

STREAMLIT UI 구성

- FastAPI에서 전체 도서 리스트를 가져와 Streamlit의 `st.table()`로 표 형태로 출력
- 각 도서는 ID, 제목, 저자, 총 권수, 대여 가능 권수 등을 한눈에 볼 수 있도록 정리하여 보여 준다.
- 목록이 비어 있는 경우에는 안내 메시지(“도서가 없습니다.”)를 표시

도서 목록

	ID	제목	저자	총 권수	대여 가능 권수
0	1	서버프로그램 기초	안철훈	2	1
1	2	SW 기초	홍길동	1	0
2	3	머신러닝 실습	홍길동	3	3

```
st.subheader("📖 도서 목록")
books = get_books()
```

📌 `Hiển thị bảng thông tin`

```
if books:
    st.table(
        [
            {
                "ID": b["id"],
                "제목": b["title"],
                "저자": b["author"],
                "총 권수": b["total_copies"],
                "대여 가능 권수": b["available_copies"],
            }
            for b in books
        ]
    )
else:
    st.info("도서가 없습니다.")
```


STREAMLIT UI 구성

- 도서 목록을 expander 형태로 표시하고, 각 도서마다 ‘삭제하기’ 버튼을 제공.
- 버튼을 누르면 Streamlit이 FastAPI의 DELETE /books/{id} 엔드포인트로 요청을 보내며, 삭제가 성공하면 페이지를 자동 새로고침하여 목록을 업데이트
- 삭제할 수 없는 경우에는 서버에서 받은 오류 메시지를 화면에 표시.

📌 도서 삭제

▼ [1] 서버프로그램 기초 - 안철훈

총 권수: 2

대여 가능 권수: 1

🗑 삭제하기

오류 발생: 이미 대여 기록이 있어서 삭제할 수 없습니다.

▶ [2] SW 기초 - 홍길동

▶ [3] 머신러닝 실습 - 홍길동

```
# 📌 Khu vực xóa sách
st.subheader("📌 도서 삭제")
```

```
if books:
    for b in books:
        with st.expander(f"[{b['id']}] {b['title']} - {b['author']}"):
            st.write(f"총 권수: {b['total_copies']}")
            st.write(f"대여 가능 권수: {b['available_copies']}")

            if st.button("🗑 삭제하기", key=f"del_{b['id']}"):
                r = requests.delete(f"{BACKEND_URL}/books/{b['id']}")
                if r.status_code == 200:
                    st.success("도서를 삭제했습니다.")
                    st.rerun() # 🔄 Refresh lại trang sau khi xóa
                else:
                    try:
                        detail = r.json().get("detail", r.text)
                    except Exception:
                        detail = r.text
                    st.error(f"오류 발생: {detail}")
```

STREAMLIT UI 구성

대여 / 반납

- 선택박스로 도서 선택
- 대여자 이름 입력
- 대여 처리, 반납 처리
- 현재 대여 중인 목록 표시

대여 중인 도서 목록

▼ [4] 홍길동 님이 도서 ID 2을(를) 대여 중

```
▼ {
  "id" : 4
  "book_id" : 2
  "borrower_name" : "홍길동"
  "borrow_date" : "2025-11-27T06:45:39.704531"
  "return_date" : NULL
  "is_returned" : false
}
```

반납

반납이 완료되었습니다.

➤ [5] ggkkj 님이 도서 ID 1을(를) 대여 중

➤ [6] 홍길동 님이 도서 ID 3을(를) 대여 중

도서관 대여 시스템

대여

대여할 도서 선택

3 - 머신러닝 실습 (3권 남음)

대여자 이름

홍길동

대여

도서 대여가 완료되었습니다.

대여 중인 도서 목록

➤ [4] 홍길동 님이 도서 ID 2을(를) 대여 중

➤ [5] ggkkj 님이 도서 ID 1을(를) 대여 중

➤ [6] 홍길동 님이 도서 ID 3을(를) 대여 중

STREAMLIT UI 구성

대여·반납 내역

- 전체 이력 테이블 표시
- 대여자 이름 필터링 기능
- 날짜, 상태 표시

도서관 대여 시스템

대여 / 반납 이력

대여자 이름으로 검색 (선택)

	대여 ID	도서 ID	대여자	대여일	반납일	상태
0	1	1	Ngo Thi Ly	2025-11-27T06:03:38.515379	2025-11-27T06:13:22.199648	반납 완료
1	2	3	Canada	2025-11-27T06:05:04.447093	2025-11-27T06:05:57.373867	반납 완료
2	3	3	Komal	2025-11-27T06:13:16.395106	2025-11-29T04:01:11.864446	반납 완료
3	4	2	홍길동	2025-11-27T06:45:39.704531	2025-11-30T12:04:39.270985	반납 완료
4	5	1	ggkkj	2025-11-27T12:49:35.991219	•	대여 중
5	6	3	홍길동	2025-11-30T12:03:50.098546	•	대여 중